

How Much of a Decision Tree Does Network Traffic Use? Leaf Locality and Demand-Paged Inference in a Programmable Switch

Philip Akekudaga

Machine Learning course project, Fall 2026

Abstract

Programmable switches can classify traffic with a decision tree at line rate, but their match-action tables hold only a small number of entries. A switch can therefore keep only a subset of a tree's leaves and forward the remaining flows to a server that runs the full tree. Whether this is useful depends on two properties of the model and the traffic: how many leaves the traffic actually uses over time, and how large a tree must be to classify well. We measure both on 22.3 million TON_IoT network flows replayed in time order. We first show that the usual random train/test split inflates the benefit of deeper trees, because 59% of test flows are exact copies of training flows. With a split that keeps every distinct feature vector in one partition, macro-F1 stops improving beyond about 650 leaves while the number of leaves the traffic uses keeps growing. On the replay stream, 2 to 18 leaves serve 90% of the flows of each attack type, while benign traffic needs 43. An online leaf cache with 8 entries, managed by decayed LFU, serves 94.7% of all flows, 1.7 points below the offline optimum (Belady) and well above the best fixed table chosen in hindsight (70.9%). We implement the scheme on BMv2 with one range-match entry per leaf and an exact integer encoding of the tree's thresholds; the prototype reproduced the full tree's predictions on all 60,000 replayed queries. On a Tofino-1 switch with real servers, 1.08 million queries were answered with the full tree's prediction every time, a switch hit took 362 ns in the pipeline, and the median round trip was about one third of sending every query to a CPU server.

1 Introduction

In-network machine learning places a trained model in the data plane of a programmable switch, so that classification happens as packets are forwarded rather than on a separate server [1][2][3][4]. Decision trees map naturally onto switch pipelines. Each leaf of a tree is an axis-aligned box of feature space, and a match-action table with one range key per feature can hold one leaf per entry. However, switch table memory is small and shared with forwarding, and a tree trained on real traffic can have thousands of leaves.

Existing systems handle this limit in two ways. The first group fits a model that is small enough to hold entirely in the switch, for example by bounding tree depth or by splitting the model across pipeline stages [3][5][6]. The second group splits the work between the switch and a server, running a small model in the data plane and a larger one behind it [1][2]. These approaches are effective for their objectives, but both fix the in-switch portion of the model before deployment. Consequently, neither can follow traffic whose mix changes over time, and neither measures how much of the full

model the traffic actually exercises.

In this project, we ask whether decision-tree inference has a working set in the sense of virtual memory [7]: a small, slowly changing subset of leaves that serves most flows. If it does, a switch can hold only the current working set and page the rest in on demand. A leaf is then a page, a switch table with K entries is the resident memory, and a flow whose leaf is absent is a page fault that the server resolves with the full tree. Because the leaves of one tree are disjoint, any subset can be resident without priorities or dependency handling, unlike general rule caching [8][9], and the switch returns exactly the full tree’s prediction whenever it answers.

We make three contributions.

- **Removes a leakage artifact from the model-size question.** TON_IoT contains millions of exact-duplicate flows. Under a random split, deeper trees appear monotonically better because they memorize duplicates. With a split that assigns each distinct feature vector to a single partition, macro-F1 stops improving beyond depth 10 to 12 while the working set keeps growing (Sections 5.1 and 5.3).
- **Measures leaf locality under time-ordered traffic.** We report the working-set size, top- K coverage, reuse distance and drift of leaf usage on 22.3 million flows, separately for the full stream, benign traffic, each attack phase and each attack type (Section 5.2). Attack flows use very few leaves; benign traffic determines the table size.
- **Evaluates demand paging, in simulation and on BMv2.** We compare online replacement policies with static placement and the offline optimum, measure adaptation after attack-phase changes in flows, and run the scheme on BMv2 and on a Tofino-1 switch with an exact encoding of the tree’s thresholds (Sections 5.4 to 5.6).

2 Background and Objectives

2.1 Decision-tree leaves as pages

A trained decision tree partitions feature space into disjoint leaf regions. For a sample x and a leaf L , x reaches L if and only if $lo_f < x_f \leq hi_f$ for every feature f , where the bounds come from the split thresholds on the path from the root to L . We call each leaf a page. A switch table holds at most K pages. A query hits if its leaf is resident and misses otherwise; on a miss, the server runs the full tree, returns the prediction and may install the leaf, evicting another one if the table is full. Because the pages are disjoint and cover the space, the answer to every query equals the full tree’s prediction. Paging changes where a prediction is computed, never what it is.

2.2 Objectives

We state four research objectives and reuse their labels throughout.

- **RO1 (prediction).** How well does a tree on features a switch can observe detect attacks when duplicate flows cannot cross the train/test boundary, compared with logistic regression, random forest and gradient boosting?
- **RO2 (locality).** Under chronological replay, how many leaves serve 90% and 99% of flows, and does the answer hold for benign traffic, for each attack phase and under different flow orderings?
- **RO3 (size and working set).** How does the working set grow with tree size, and where does the accuracy peak fall relative to it?

- **RO4 (adaptation).** With K resident leaves, how close do online replacement policies come to the offline optimum, how much better are they than static placement, and how quickly do they recover after an attack-phase change?

2.3 Scope

We assume that flow features are computed before classification and carried to the switch in a custom header, as in the standard workflow of in-network classification prototypes [4]. We argue that this is a reasonable assumption for a study of model placement, as line-rate feature extraction is a separate problem with its own resource costs. We run the scheme on BMv2, a software switch, and on a Tofino-1 switch with two servers. Round trips measured at a server include its network stack, so we report the on-chip latency separately, and we make no line-rate claims.

3 Design

3.1 Exact range encoding of leaves

To install a leaf as one table entry, each feature must be matched as an integer range. The tree compares float32 inputs with float64 thresholds, so a naive integer quantization can move a sample across a threshold. To keep the switch’s answers identical to the tree’s, we send each feature as a 32-bit key that preserves float32 order: for a non-negative value we set the sign bit of its IEEE-754 bit pattern, and for a negative value we invert all bits. For a leaf bound hi , the matching key bound is the key of the largest float32 value not exceeding hi ; for lo it is the key of the smallest float32 value above lo . A range match on keys then reproduces every comparison of the tree, including samples that lie exactly on a threshold. Unit tests check the encoding against the tree’s own leaf assignment on random data and at every split threshold.

3.2 Replacement policies

We compare four online policies, a static baseline and two references. **LRU** evicts the least recently used leaf. **LFU** evicts the leaf with the fewest accesses over the whole history. **Decayed LFU** keeps an exponentially decayed score $S_i(t) = \gamma S_i(t-1) + \mathbb{1}[L_t = i]$ and evicts the lowest score; we evaluate $\gamma \in \{0.99, 0.999\}$. To avoid numerical underflow over millions of flows, we order residents by the time-free key $\log S_i - t_i \log \gamma$, which ranks leaves exactly as $S_i \gamma^{t-t_i}$ does. **Static top-K** installs the most frequent leaves of the stream’s first hour and never changes. The **hourly oracle** preloads each hour’s true top-K leaves; it is not deployable. **Belady’s algorithm** [10], extended to bypass a missed leaf that is needed later than every resident, is the offline optimum for any policy that changes at most one entry per miss, and it bounds the online policies. The best fixed table in hindsight, $C(K)$, is the share of flows served by the K most frequent leaves of the whole stream.

3.3 Prototype

In Figure 1, we show the prototype.

The P4 program holds a range-match table with one entry per resident leaf and ten 32-bit range keys. The controller sends each flow’s feature keys as a P4Runtime packet-out. The switch looks the keys up and returns a packet-in with the query identifier, a hit flag, the class and the leaf identifier. On a miss, the controller runs the full tree, updates the replacement policy and applies

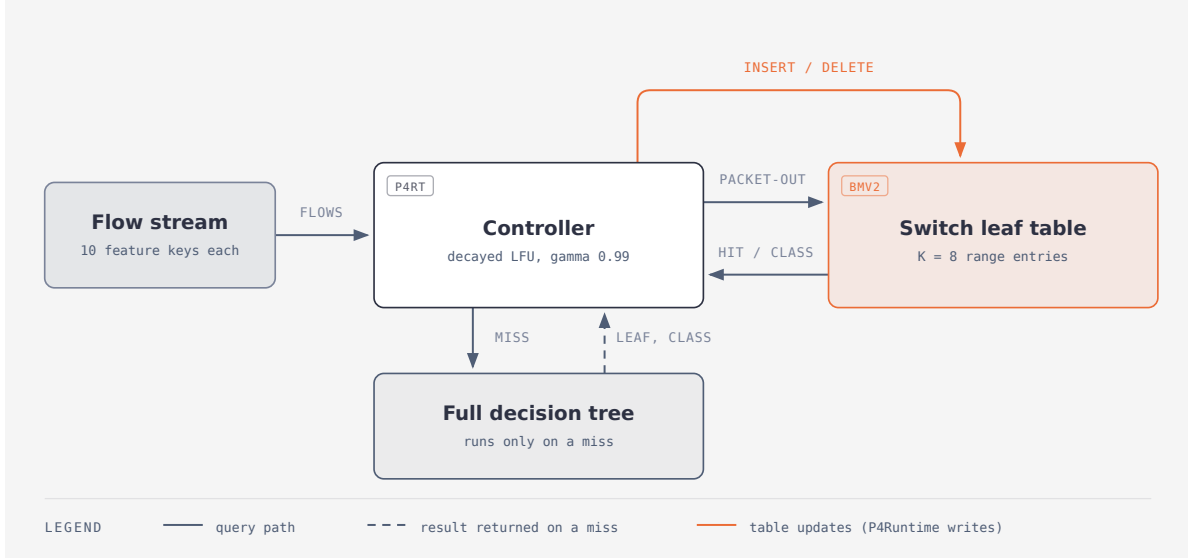


Figure 1: MVM-Lite prototype. The controller sends feature keys as packet-outs; the switch answers from its 8-entry leaf table; on a miss the controller runs the full tree and rewrites the table.

the resulting deletions and insertions in P4Runtime writes. Because queries enter and leave through the controller channel, the prototype needs no network interfaces.

3.4 Encoding for Tofino-1

Tofino-1 limits range keys to about 20 bits per table, so the ten 32-bit range keys of the BMv2 table do not fit. We therefore split the lookup in two stages. For each feature, two static tables write a thermometer code with one bit per distinct threshold of that feature in the tree: bit j is set when the value exceeds threshold j . A coarse table matches the top 20 bits of the feature key as a range; a fine table, applied first, matches the few 4,096-key buckets that contain a threshold exactly on the top 20 bits and as a 12-bit range on the rest. The leaf table is ternary over the concatenated codes. Because a thermometer code is monotone, a leaf box $lo < x \leq hi$ needs only two cared-for bits per feature, bit(lo) set and bit(hi) clear, so each leaf is still exactly one entry. The key width equals the number of distinct (feature, threshold) pairs: 361 bits for the depth-10 tree and 602 bits for depth 12, which exceeds the ternary key budget. We therefore deploy the depth-10 tree (394 leaves), which lies on the Pareto frontier of Section 5.3. The encoding reproduces the tree’s leaf assignment exactly, which unit tests check at every split threshold.

4 Methodology

Data. We use the processed network flows of TON_IoT [11], 22,339,021 Zeek flow records with timestamps at one-second resolution. We drop 869 malformed records whose byte counter holds an IP address. The stream is 96.4% attack traffic, and the attacks run in day-long phases (Canberra local time): scanning on 23 to 24 April 2019, denial of service and injection on 25 April, distributed denial of service on 26 April, password and cross-site scripting attacks on 27 April, ransomware and backdoor on 28 April, and backdoor and man-in-the-middle on 29 to 30 April, preceded by three days of benign traffic in early April. The official training sample of the dataset [12] was not available to us, so we draw training pools with the same class mix (300,000 benign flows and 20,000

flows per attack type) from the full set.

Splits. A random split of TON_IoT leaks, because floods produce millions of identical feature vectors [13]. We use three splits. The *grouped* split hashes each distinct data-plane feature vector, mixed with the seed, to exactly one of train, validation and test (60/20/20); it is our primary IID evaluation. The *forward* split is grouped and additionally draws pools only from the first half of each day, replaying only the second halves; injection flows lie entirely in the second half of 25 April, so injection is an unseen attack in this split. The *random* split samples rows independently and is kept only to measure how much duplicates inflate the scores. Every pooled row is removed from the replay stream.

Features and models. The main feature set contains what a switch can count or parse: duration, source and destination bytes, packets and IP-layer bytes, missed bytes, protocol and connection state. IP addresses and ports are excluded, because in this testbed they identify attacker hosts; an ablation adds them back. Categorical values are coded by the rank of their training attack rate. We train logistic regression, decision trees (maximum depth 4 to unlimited, and a cost-complexity pruning path), random forests and histogram gradient boosting. Hyperparameters are chosen on validation macro-F1 only, and every result is averaged over five seeds.

Metrics. For classification we report macro-F1, MCC, per-class recall and PR-AUC, together with the Bayes ceiling implied by identical vectors that carry both labels. For locality we report the working set W_q , the fewest leaves that serve a fraction q of flows; the top-K coverage $C(K)$; normalized leaf entropy; reuse distance, from which the LRU hit rate follows exactly for every K ; and the hourly Jensen-Shannon divergence of the leaf distribution. For caching we report hit rate, churn (insertions plus evictions per flow), the gap to Belady, and adaptation after each attack-phase change measured in flows. Replay metrics are serially dependent, so their intervals come from a block bootstrap over hours.

5 Evaluation

5.1 Effectiveness in RO1: prediction without duplicate leakage

Because TON_IoT floods repeat the same flow record many times, the choice of split decides what the test set measures. Under the random split, 59% of test flows have an exact copy in the training pool, and every model scores between 0.866 and 0.967 macro-F1. With the grouped split, only 1% of test vectors appear in training. The unpruned tree then scores 0.918 ± 0.040 , the random forest 0.919 ± 0.041 , gradient boosting 0.909 ± 0.017 and logistic regression 0.780 (means over five seeds).

Consequently, the random split overstates the tree by 0.049 and logistic regression by 0.086. The Bayes ceiling of the grouped test set is 0.970: 3% of its flows share a feature vector with flows of the other class, so no model can classify them all.

The feature ablation separates what the model learns from what the testbed leaks. Adding the Zeek service, DNS, HTTP and SSL fields to the data-plane features changes the tree’s macro-F1 by less than 0.01 (0.916 against 0.918). Adding IP addresses and ports raises it to 0.9999. We suspect the reason is that the attacks in this testbed come from a small set of hosts, ten Kali Linux machines at 192.168.159.30 to 192.168.159.39 according to the dataset documentation [11], and the tree learns their addresses rather than their behavior. We therefore exclude addresses and ports from every other experiment.

The IID test set is balanced by attack type, but the replay stream is 96% attack traffic. On the

forward split, where training uses only the first half of each day, the validation-selected tree reaches 0.624 macro-F1 on the chronological replay, against 0.491 for a classifier that labels every flow an attack; gradient boosting reaches 0.827. Injection never appears in training under this split, and the tree still flags 51% of injection flows.

The gap between IID and replay scores appears to come mostly from benign precision: at a 96% attack share, a small fraction of attack flows misclassified as benign outnumbers the true benign flows. We treat the replay numbers as the deployment-relevant ones and report both.

5.2 Effectiveness in RO2: leaf locality

We replay the validation-selected tree (unpruned, 1,880 leaves on average) over the 21.7 million flows of the grouped replay stream in time order. In Figure 2, we show the leaf popularity. Leaf usage is highly skewed: 26 leaves serve 90% of all flows and 266 serve 99%. The skew differs by traffic type.

For each attack type except man-in-the-middle, between 1.6 leaves (backdoor, denial of service) and 17.8 leaves (injection) serve 90% of its flows, and scanning needs 4.6. Benign traffic needs 43 leaves, and the rare man-in-the-middle flows (527 in the stream) spread over 96. Consequently, attack floods are cheap to cache, and the table size is set by benign traffic.

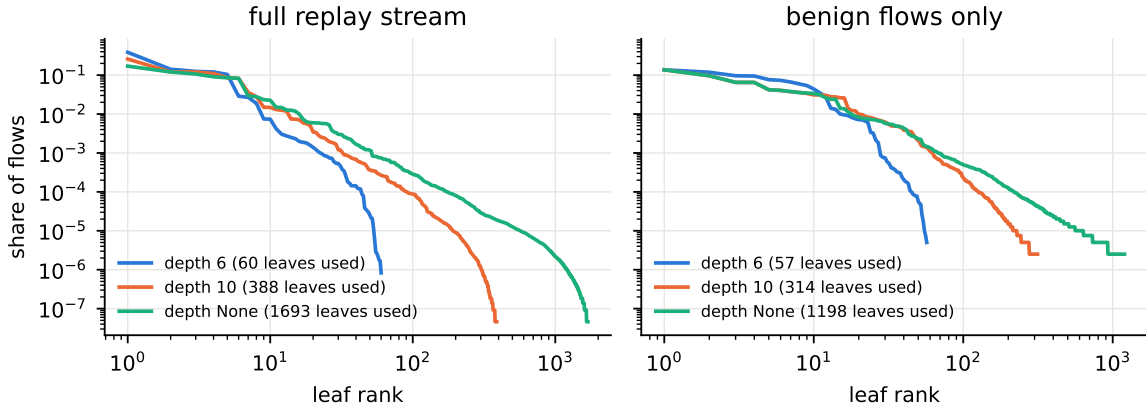


Figure 2: Leaf popularity on the grouped replay stream (seed 0) for trees of depth 6, 10 and unpruned; left: all flows, right: benign flows only.

Frequency alone does not explain the locality. The best fixed table of 8 leaves chosen in hindsight serves 70.9% of all flows, whereas an 8-entry LRU cache serves 93.6%, and on benign traffic the gap is 57.7% against 82.1%. The reason is that the popular leaves change from one attack phase to the next, and an online cache follows them. The hourly Jensen-Shannon divergence of the leaf distribution stays small within a phase and jumps at phase changes, which is the drift that RO4 addresses.

Flow order within a second is not recorded by the dataset, so we tested three orderings. Shuffling flows within each second changes the 8-entry LRU hit rate by less than 0.001. Ordering flows by their end time, when a switch could first classify a completed flow, raises the 1-entry hit rate from 0.51 to 0.77 but moves the 8-entry hit rate only from 0.936 to 0.948. The conclusions for tables of 8 or more entries therefore hold under all three orderings.

5.3 Effectiveness in RO3: tree size and working set

In Figure 3, we show macro-F1 against tree size on the grouped split. Accuracy rises from 0.836 at depth 4 (16 leaves) to 0.904 at depth 10 (395 leaves) and then stops improving: depth 12 scores 0.921 ± 0.027 with 650 leaves, and the unpruned tree scores 0.918 ± 0.040 with 1,880 leaves. The differences beyond depth 10 are smaller than the variation across seeds. Under the random split, by contrast, macro-F1 rises monotonically from 0.863 to 0.967, because deeper trees memorize more duplicated flows. The apparent benefit of an unpruned tree is therefore a property of the leaky evaluation, not of the model.

The working set does not stop growing. The number of leaves that serve 90% of benign flows is 6 at depth 4, 24 at depth 10 and 43 for the unpruned tree, and the number that serves 99% of all flows grows from 10 to 266.

In Figure 4, we plot macro-F1 against the benign working set. Depths 4, 6, 10 and 12 lie on the Pareto frontier, while depth 16 and the unpruned tree are dominated: they cost more table entries without a measurable accuracy gain. Validation selection is unstable in this regime; the five seeds selected the unpruned tree twice and depths 10, 12 and 16 once each. Consequently, a switch designer can bound the tree at depth 12 without giving up accuracy that the evaluation can resolve.

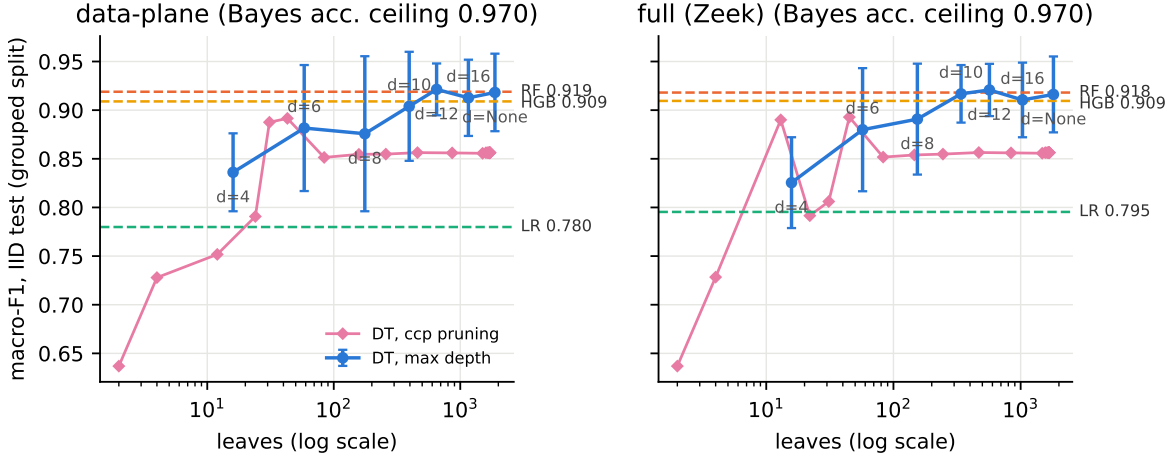


Figure 3: Macro-F1 on the grouped IID test set against the number of leaves (log scale), for maximum-depth trees (mean $\pm$ sd over five seeds) and the cost-complexity pruning path; dashed lines mark the other model families.

5.4 Effectiveness in RO4: demand paging

In Figure 5, we compare replacement policies across table sizes for the depth-10 tree; the ranking of the policies is the same for the validation-selected unpruned tree, whose numbers we quote here. On the full stream with 8 entries, decayed LFU with $\gamma = 0.99$ serves 94.7% of flows, 1.7 points below Belady’s offline optimum (96.4%). LRU serves 93.6% and the hourly oracle 92.9%, which confirms that the oracle, although it knows each hour’s top leaves, is not an upper bound: online policies adapt within the hour.

Plain LFU serves 77.8% and writes 0.445 table entries per flow, against 0.106 for decayed LFU, because it keeps leaves that were popular in earlier phases. A static table filled from the first hour

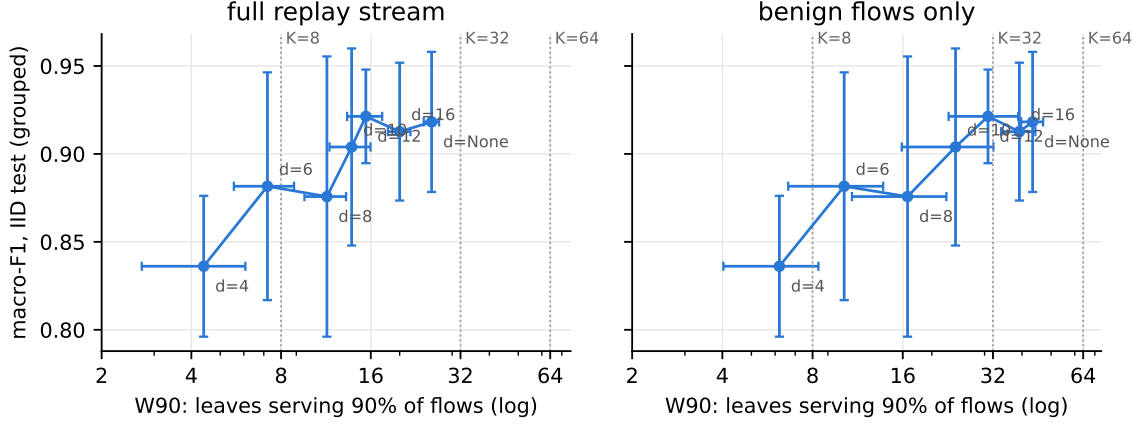


Figure 4: Macro-F1 against the working set W90 on the full stream (left) and on benign flows (right); dotted lines mark table sizes of 8, 32 and 64 entries.

of the stream serves 1.3%, because that hour is benign traffic weeks before any attack; this is the failure mode of placing leaves before deployment. On benign traffic alone, decayed LFU serves 84.8% against 89.8% for Belady.

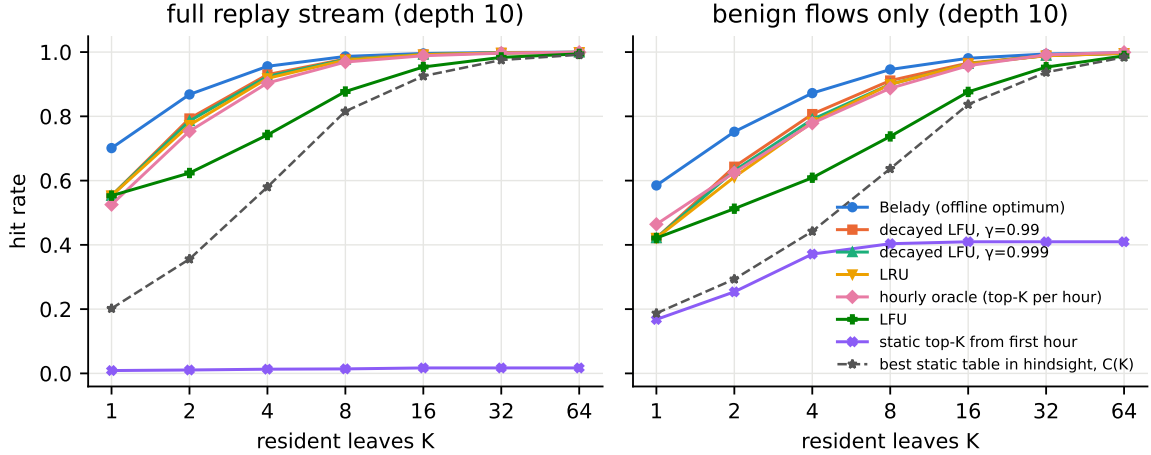


Figure 5: Hit rate against table size K for each replacement policy (grouped split, depth-10 tree, mean over five seeds); the dashed line is the best fixed table in hindsight, $C(K)$.

To measure adaptation, we count misses at every attack-phase boundary, i.e., every day whose set of attack types differs from the previous day's, with the tree at depth 10 and 8 entries. In the first 10,000 flows of a new phase, decayed LFU incurs on average 119 misses more than its steady-state rate (at most 673), LRU 133 (at most 1,010) and LFU 836 (at most 3,703).

Recovery takes 1,500 flows or fewer at every boundary except 27 April, where password and cross-site scripting attacks start together and even Belady needs about 16,000 flows; the long transient there is a property of the phase, not of the policy. As shown in Figure 6, the hit rate of the adaptive policies drops briefly at each phase change and returns, while the static table never recovers.

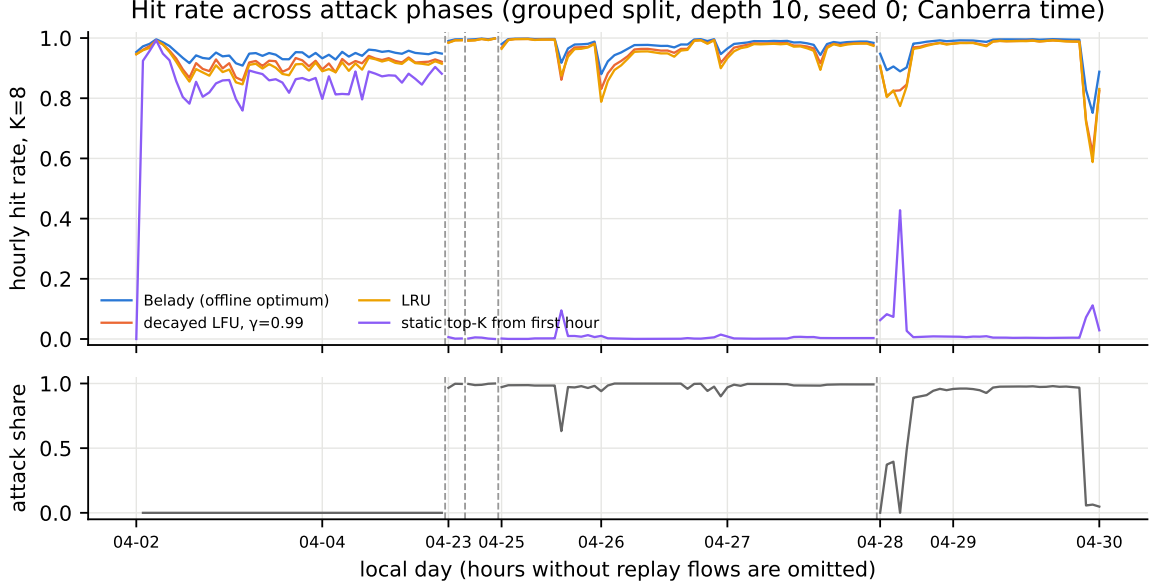


Figure 6: Hourly hit rate with 8 entries across the attack phases (grouped split, depth 10, seed 0; top) and the attack share of each hour (bottom).

5.5 Prototype on BMv2

We ran the validation-selected tree of seed 0 (unpruned, 1,720 leaves) on BMv2 with an 8-entry table and decayed LFU in the controller. Each of two slices contains 30,000 consecutive flows of the replay stream around a phase change: all traffic across 25 to 26 April, and benign traffic across 27 to 28 April. Table 1 summarizes the checks and measurements. The served prediction equaled the full tree’s prediction for all 60,000 queries. The switch hit exactly when the controller’s policy held the query’s leaf, and the hit sequence was identical to the offline simulator, with hit rates of 92.2% on the full slice and 85.6% on the benign slice. The table never held more than 8 entries.

Table 1: Prototype checks and measurements on BMv2 (8 entries, decayed LFU, 30,000 queries per slice). Latencies are measured by the controller and reflect a software switch.

	full stream, 25 to 26 April	benign flows, 27 to 28 April
served prediction equals full tree	100%	100%
switch hit sequence equals simulator	yes	yes
hit rate	92.2%	85.6%
hit time, median / 99th percentile	0.21 / 0.28 ms	0.24 / 0.31 ms
miss service time, median / 99th percentile	0.85 / 1.20 ms	1.02 / 1.31 ms
miss: full tree / table writes (median)	0.13 / 0.49 ms	0.17 / 0.60 ms

On BMv2, a hit took 0.21 ms at the median (0.28 ms at the 99th percentile), measured by the controller from packet-out to packet-in. A miss took 0.85 ms at the median, of which 0.13 ms was the full tree and 0.49 ms the P4Runtime writes. BMv2 accepted 3,866 single-entry insertions per second and 33,622 per second in batches of 100. With decayed LFU writing about 0.11 entries per flow on the full stream, single-entry writes would sustain roughly 35,000 flows per second on this software switch; batching or a hardware control plane would change that figure, and we do not extrapolate it to an ASIC.

5.6 Tofino-1 hardware

We deployed the encoding on a UfiSpace S9180-32X switch (Tofino-1, SDE 9.13.2) with two servers on 25 Gb/s links. Vision sends queries into the switch. A hit is answered by the switch and reflected back to Vision; a miss is forwarded to Hulk, whose C program runs the full depth-10 tree on the CPU and replies through the switch, while a digest informs the controller on the switch CPU. The controller runs decayed LFU with 8 resident leaves. Because hits never reach the controller, it credits resident leaves from their hit counters every 20 ms. As a CPU-only baseline, we set K to 0, so every query takes the Hulk path through the same switch and links. The compiled program uses all 12 ingress stages, with the leaf table in the last one. Each configuration ran three times from a cold cache on the two 30,000-flow slices of Section 5.5.

All 36 runs answered all 1,080,000 queries, and the served class and leaf equaled the offline tree for every query. At 1,000 queries per second, the switch served 95.6% of the full-stream slice and 84.6% of the benign slice, against 96.3% and 88.5% for the ideal decayed LFU of the simulator, which BMv2 matched exactly. The gap comes from control-plane lag: the controller needed 6.1 ms at the median (24.4 ms at the 99th percentile) from a digest to a completed table write, and a simulator of the implementable policy with a lag of 10 to 50 queries brackets both hardware hit rates.

Table 2 and Figure 7 compare the latencies. A switch hit took 362 ns from ingress to egress in the pipeline (391 ns at the 99th percentile). Measured at Vision, a hit returned after 102.5 μ s at the median, a miss answered by Hulk after 232 μ s, and a query in the CPU-only configuration after 300.7 μ s. The CPU computation itself is not the cost: Hulk classifies a query in about 30 ns in memory. The difference comes from the extra network hop and the host network stacks, which also dominate the hit round trip: at 100,000 queries per second the hit round trip fell to 32.5 μ s, which we attribute to interrupt coalescing on the hosts rather than to the switch.

Table 2: Latency on the Tofino-1 testbed at 1,000 queries/s (full-stream slice, three runs pooled).

path	median	99th percentile
switch pipeline, hit (on-chip timestamps)	362 ns	391 ns
round trip at Vision, switch hit (MVM)	102.5 μ s	133.0 μ s
round trip at Vision, miss answered by Hulk (MVM)	232.2 μ s	324.7 μ s
round trip at Vision, CPU only (K=0)	300.7 μ s	338.1 μ s

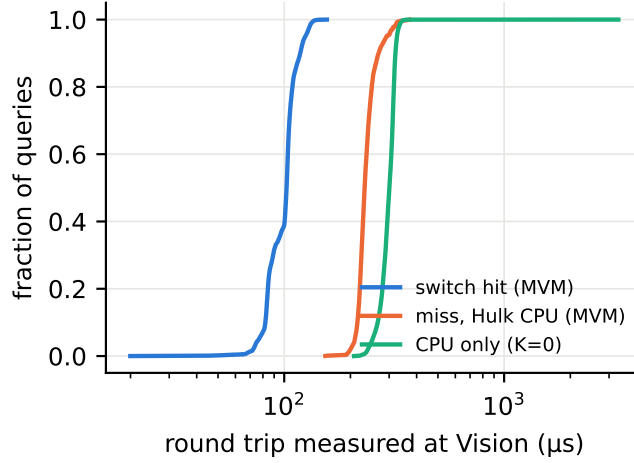


Figure 7: Round trip measured at Vision at 1,000 queries/s on the Tofino-1 testbed: switch hits, misses answered by Hulk, and the CPU-only configuration.

As shown in Figure 8, both configurations answered every query up to 100,000 queries per second, and MVM’s median round trip stayed two to three times lower than the CPU-only configuration at every load. The switch hit rate, however, fell from 95.6% at 1,000 queries per second to between 67.6% and 80.9% at 5,000 to 100,000 queries per second, and the decline was not monotone. The reason is that a fixed control-plane delay spans more queries as the rate grows, so more repeats of a newly missed leaf arrive before its entry is installed; the non-monotone shape may also reflect runs at the highest rates that end within a few hundred milliseconds, before the controller has reacted. Consequently, the replacement policy that works on the simulator needs a faster control path, or decisions made in the data plane, to hold its hit rate at high load.

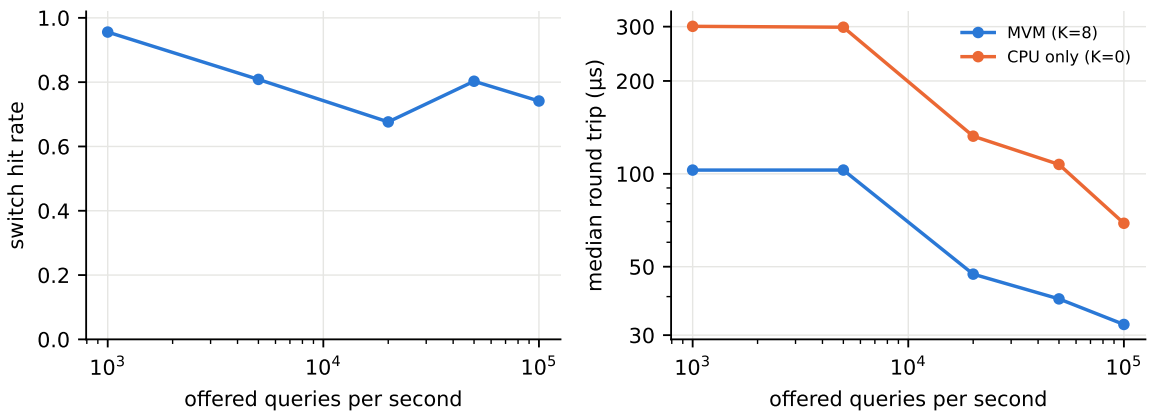


Figure 8: Load sweep on the Tofino-1 testbed (full-stream slice, mean of three runs). Left: switch hit rate of MVM. Right: median round trip at Vision for MVM and the CPU-only configuration.

6 Related Work

In-network classification. IIsy [1] runs a small model in the switch and a larger model on a backend, and updates models through table writes. NetBeacon [2] classifies flows in several phases and stops early once confident. Leo [3] runs decision trees at multi-terabit rates on Tofino and reprograms them at runtime without downtime, and SpliDT [5] splits a tree into subtrees to reuse pipeline resources. Planter [4] and Mousika [6] map trained models onto switch targets. These systems decide the in-switch model before deployment or swap whole models. Compared to them, MVM keeps part of a single tree resident and selects that part from the traffic, and we measure how large that part needs to be.

Rule caching. CacheFlow [8] caches popular forwarding rules in the switch and keeps the rest in software, and CAB [9] partitions the rule space into buckets for the same purpose. Both must handle dependencies between overlapping rules of different priority. The leaves of one decision tree never overlap, so MVM avoids that machinery; this property does not hold for tree ensembles, whose leaves overlap across trees.

Locality in tree inference. PACSET [14] and cache-friendly tree layouts [15] place frequently visited nodes together in memory, using leaf frequencies measured offline. Clipper [16] caches predictions for repeated queries. Our measurement differs in that we follow leaf usage over time in a traffic stream and ask how many leaves a bounded table needs, which also connects model selection to the working-set concept [7].

7 Conclusion

We measured how much of a decision tree real network traffic uses and whether a switch can hold only that part. Under a split without duplicate leakage, a tree of about 650 leaves classifies as well as the unpruned tree of 1,880 leaves, while the unpruned tree needs almost twice as many leaves to serve 90% of benign flows. Attack flows use 2 to 18 leaves per type and benign traffic 43, and an 8-entry table managed by decayed LFU serves 94.7% of all flows, within 1.7 points of the offline optimum. The BMv2 prototype and the Tofino-1 deployment reproduced the full tree’s predictions on every replayed query, and on Tofino-1 a switch hit took 362 ns in the pipeline. Future work should repeat the study with the official training sample and on a second dataset, and move the replacement decisions closer to the data plane, since control-plane lag cost up to 28 points of hit rate at high load.

References

- [1] C. Zheng et al., “IIsy: Hybrid in-network classification using programmable switches,” *IEEE/ACM Transactions on Networking*, 2024 (arXiv:2205.08243).
- [2] G. Zhou et al., “An efficient design of intelligent network data plane,” *USENIX Security Symposium (NetBeacon)*, 2023.
- [3] S. Jafri et al., “Leo: Online ML-based traffic classification at multi-terabit line rate,” *USENIX NSDI*, 2024.
- [4] C. Zheng et al., “Planter: Rapid prototyping of in-network machine learning inference,” *arXiv:2205.08824*.

- [5] M. Parvez et al., “SpliDT: Partitioned decision trees for scalable stateful inference at line rate,” arXiv:2509.00397, 2025.
- [6] Mousika, IEEE INFOCOM, 2022. [authors and title to be verified]
- [7] P. J. Denning, “The working set model for program behavior,” Communications of the ACM, 1968.
- [8] N. Katta et al., “CacheFlow: Dependency-aware rule-caching for software-defined networks,” ACM SOSR, 2016.
- [9] B. Yan et al., “CAB: A reactive wildcard rule caching system for software-defined networks,” ACM HotSDN, 2014.
- [10] L. A. Belady, “A study of replacement algorithms for a virtual-storage computer,” IBM Systems Journal, 1966.
- [11] N. Moustafa, TON_IoT datasets, UNSW Canberra, <https://research.unsw.edu.au/projects/toniot-datasets>.
- [12] TON_IoT network train/test sample description, arXiv:2007.05922. [authors to be verified]
- [13] Label-noise and duplicate analysis of TON_IoT and NF-ToN-IoT, arXiv:2212.13994. [authors to be verified]
- [14] H. Madhyastha et al., “PACSET: Probabilistic assignment of tree nodes in memory,” arXiv:2011.05383, 2020. [title to be verified]
- [15] K. Gupta and B. Johnston, “Growing cache friendly decision trees,” SysML, 2018.
- [16] D. Crankshaw et al., “Clipper: A low-latency online prediction serving system,” USENIX NSDI, 2017.